TechTicket

Architecture technique

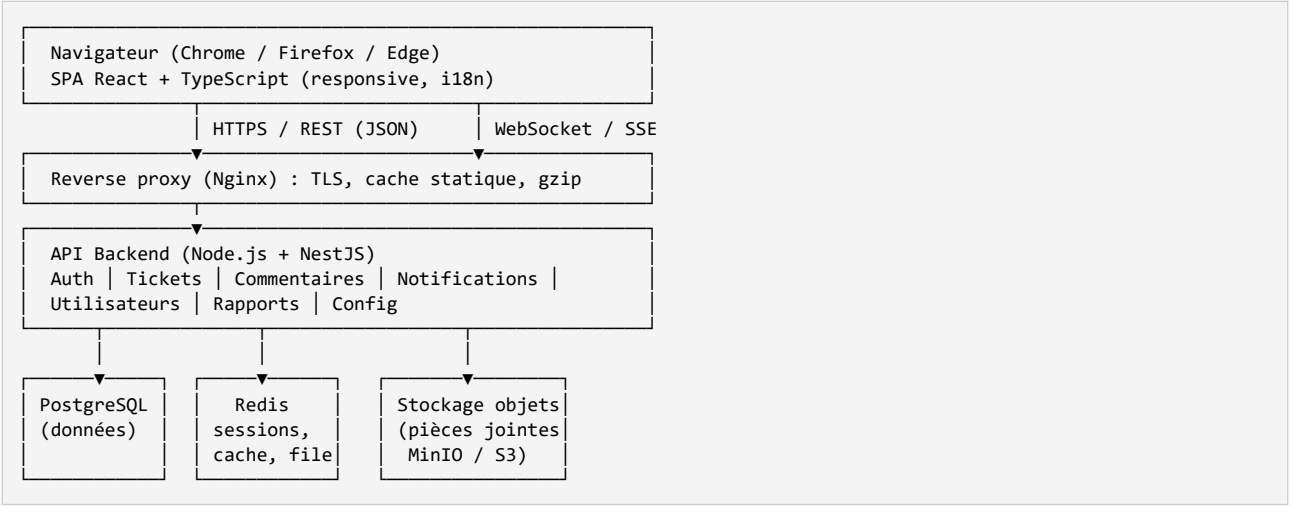
Plateforme de gestion de tickets d'assistance technique

Document établi à partir du cahier des charges fonctionnel IHM

Octobre 2026

1. Vue d'ensemble

L'architecture proposée est un **monolithe modulaire**, adapté à la taille du projet (entreprise intermédiaire, 3 rôles, 5 pages). Il est plus simple à développer et à déployer que des microservices.



2. Choix technologiques

Couche	Choix	Justification
Frontend	React + TypeScript + Vite	Composants réutilisables, bon écosystème
UI	Tailwind + shadcn/ui (ou MUI)	Cohérence visuelle (palette, icônes unifiées)
Données client	TanStack Query	Cache, rafraîchissement des statuts
Graphiques	Recharts	Barres et camembert de la page Rapports
i18n	i18next	Français par défaut, autres langues ajoutables
Backend	NestJS (Node + TS)	Architecture modulaire, guards pour les rôles, validation intégrée
ORM	Prisma	Migrations, typage fort
Base de données	PostgreSQL	Relationnel, agrégats pour les statistiques
Cache / sessions	Redis	Sessions, cache des stats, file de tâches
Fichiers	MinIO (compatible S3)	Pièces jointes hors base
Temps réel	SSE ou WebSocket	Notifications in-app, mise à jour des statuts
Déploiement	Docker Compose, puis Kubernetes si besoin	Simple à installer

Alternative plus légère : Django + DRF avec templates ou HTMX, si l'équipe maîtrise Python.

3. Modèle de données

```
User(id, nom, email, password_hash, role[EMPLOYE|TECH|ADMIN], actif, créé_le)
Category(id, nom, actif)
Priority(id, libellé, niveau, délai_cible)
Ticket(id, numéro [TT-2026-00001], titre, description,
       status, priority_id, category_id,
       demandeur_id, assigné_id, échéance,
       résolu_le, solution, créé_le, maj_le)
Comment(id, ticket_id, auteur_id, contenu, interne:boolean, créé_le)
Attachment(id, ticket_id, comment_id?, clé_stockage, nom, taille, mime)
Notification(id, user_id, ticket_id, type, lu:boolean, créé_le)
Feedback(id, ticket_id, note 1-5, commentaire)
TicketEvent(id, ticket_id, acteur_id, type, ancien, nouveau, date)  -- audit
```

Points qui répondent à des exigences précises du cahier des charges :

- **Comment.interne** : gère les notes internes, invisibles de l'employé.
- **Ticket.résolu_le** : permet de vérifier la fenêtre de réouverture de 72 h.
- **TicketEvent** : historique, audit et calcul des délais moyens de résolution.
- **Category** et **Priority** : tables configurables par l'administrateur.

Cycle de vie des statuts : OUVERT → EN_COURS ↔ EN_ATTENTE_RETOUR → RÉSOLU → FERMÉ. Un ticket peut repasser de RÉSOLU à OUVERT pendant 72 h.

4. API REST (extrait)

```
POST   /auth/login | /auth/logout | GET /auth/me

GET    /tickets?status=&priority=&assignee=&q=&page=   (filtré selon le rôle)
POST   /tickets                                       (EMPLOYE)
GET    /tickets/:id
PATCH  /tickets/:id          statut, priorité, échéance (TECH/ADMIN)
POST   /tickets/:id/assign   prise en charge / réattribution (TECH/ADMIN)
POST   /tickets/:id/resolve  avec description de la solution
POST   /tickets/:id/reopen   refusé si > 72 h
POST   /tickets/:id/comments {contenu, interne}
POST   /tickets/:id/attachments
POST   /tickets/:id/feedback

GET    /notifications | PATCH /notifications/:id/read
GET    /stats/summary | /stats/by-category | /stats/resolution-times
GET    /reports/export?format=csv|pdf                 (ADMIN)

CRUD   /users, /categories, /priorities               (ADMIN)
```

5. Sécurité

- **Authentification :** mot de passe haché (argon2 ou bcrypt), session par cookie HttpOnly + Secure + SameSite stockée dans Redis. Alternative : JWT court + refresh token.
- **Autorisation (RBAC) :** un guard NestJS vérifie le rôle sur chaque route. Le cloisonnement est appliqué côté serveur dans les requêtes (un employé ne voit que demandeur_id = moi). Le frontend ne sert qu'à l'affichage.
- **Notes internes :** exclues de la requête pour les employés, pas seulement masquées dans l'interface.
- **Pièces jointes :** liste blanche de types MIME, taille maximale, renommage, analyse antivirus (ClamAV) si possible, téléchargement par URL signée.
- **Autres mesures :** validation des entrées (DTO + class-validator), rate-limiting sur le login, protection CSRF, en-têtes de sécurité (Helmet), HTTPS obligatoire.

6. Performance et disponibilité

- **Objectif < 2 s pour 95 % des requêtes :** index sur (status, priority_id, assigné_id, créé_le), pagination côté serveur, cache Redis des stats (TTL 1 à 5 min), découpage du JS par route.
- **Disponibilité de 99,5 % :** redémarrage automatique des conteneurs, sauvegardes PostgreSQL quotidiennes (pg_dump + WAL), health checks, supervision (Prometheus + Grafana ou équivalent), logs centralisés.
- **Tâches asynchrones** (BullMQ sur Redis) : exports PDF/CSV, envoi d'e-mails, rappels d'échéance.

7. Notifications

Le cahier des charges demande des notifications in-app : événement métier → création d'une **Notification** en base → diffusion par SSE au client connecté. Un canal e-mail pourra s'ajouter plus tard sans changer le modèle.

8. Structure du dépôt

```
techticket/
├─ apps/
│  ├─ web/            React (pages: dashboard, submit, manage, track, reports)
│  └─ api/            NestJS (modules: auth, tickets, comments, ...)
├─ packages/shared/   types et enums partagés (statuts, rôles)
├─ prisma/            schema.prisma + migrations
├─ docker-compose.yml (web, api, postgres, redis, minio, nginx)
└─ .github/workflows/ CI : lint, tests, build, déploiement
```

9. Qualité et CI/CD

- **Tests :** unitaires (Jest/Vitest), intégration API (Supertest, avec une vraie base en conteneur), E2E (Playwright) sur les parcours clés : créer un ticket, le traiter, le rouvrir.
- **CI :** lint, typecheck, tests et build à chaque PR, puis déploiement automatisé sur un environnement de recette.

10. Décisions à trancher

- **Hébergement :** cloud, serveur interne ou on-premise. Cela fixe le choix du stockage de fichiers et des sauvegardes.
- **Authentification :** comptes locaux (comme dans le cahier des charges) ou SSO (Azure AD, LDAP) à terme.
- **Notifications :** in-app seulement, ou aussi e-mail.
- **Équipe :** si elle est plus à l'aise avec Python ou Java, l'architecture reste valable en changeant le backend (Django ou Spring Boot).